FUNCTIONS

ALVINA ANJUM



PREVIOUS LECTURE

- Pointers
- Arrays and Pointers
- Double Pointers

LECTURE OUTLINE

- Introduction to Functions
- Defining Functions
- Calling Function
- Function Prototype
- Void Functions

FUNCTIONS

- A program may be broken up into manageable functions.
  - A function is a collection of statements that performs a specific task.
    1) We have created a function named **main** in every program we've written
    2) We have used library functions such as **strlen, pow**
- Functions are commonly used to break a problem down into small manageable pieces.

This program has one long, complex function containing all of the statements necessary to solve a problem.

```
int main()
{
    statement;
    statement;
    statement;
    statement;
    statement;
    statement;
    statement;
    statement;
    statement;
    statement;
    statement;
    statement;
    statement;
    statement;
    statement;
    statement;
    statement;
    statement;
    statement;
    statement;
    statement;
    statement;
    statement;
}
```

In this program the problem has been divided into smaller problems, each of which is handled by a separate function.

```
int main()
{
    statement;
    statement;
    statement;
}
```

main function

```
void function2()
{
    statement;
    statement;
    statement;
}
```

function 2

```
void function3()
{
    statement;
    statement;
    statement;
}
```

function 3

```
void function4()
{
    statement;
    statement;
    statement;
}
```

function 4

FUNCTIONS PROVIDE REUSABILITY

- Another reason to write functions is that they simplify programs
- If a specific task is performed in several places in a program, a function can be written once to perform that task, and then be executed anytime it is needed.
- For example: We need to find the sum of integers from 1 to 10, from 20 to 37, and from 35 to 49, respectively.

REUSABILITY: EXAMPLE

```
for (int i = 1; i <= 10; i++)
  sum += i;
cout << "Sum from 1 to 10 is " << sum << endl;

sum = 0;
for (int i = 20; i <= 37; i++)
  sum += i;
cout << "Sum from 20 to 37 is " << sum << endl;

sum = 0;
for (int i = 35; i <= 49; i++)
  sum += i;
cout << "Sum from 35 to 49 is " << sum << endl;
```

```
int sum(int i1, int i2)
{
  int sum = 0;
  for (int i = i1; i <= i2; i++)
    sum += i;

  return sum;
}

int main()
{
  cout << "Sum from 1 to 10 is " << sum(1, 10) << endl;
  cout << "Sum from 20 to 37 is " << sum(20, 37) << endl;
  cout << "Sum from 35 to 49 is " << sum(35, 49) << endl;

  return 0;
}
```

DEFINING FUNCTION

- A function definition contains the statements that make up the function.
- *A function definition consists of its function name, parameters, return value type,and body.*
- The syntax for defining a function is as follows:

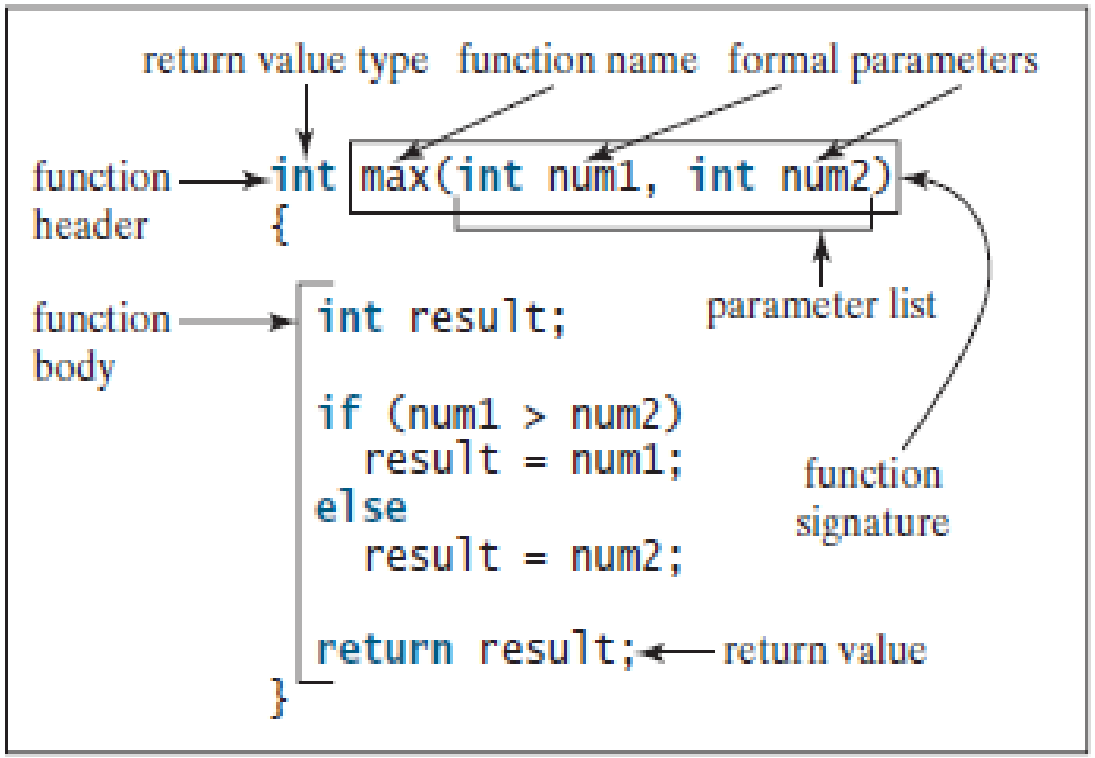
```
returnValueType functionName(list of parameters)
{
  // Function body;
}
```

DEFINING FUNCTION: EXAMPLE

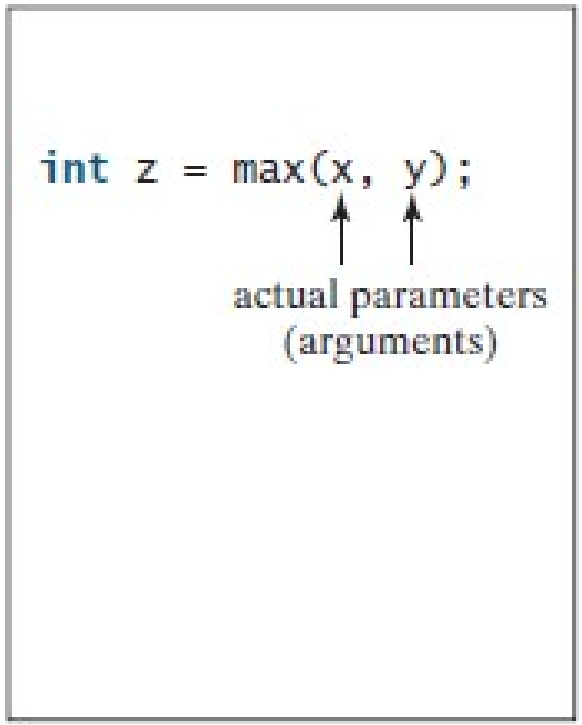
- Let's look at a function created to find which of two integers is bigger.

cout<<"max num is:"<<z;

Define a function



Invoke a function



CAUTION

- In the function header, you need to declare each parameter separately. For instance, **int max(int num1, int num2)** is **correct**, but **max(int num1, num2)** is **wrong.**

CALLING A FUNCTION

- *Calling a function executes the code in the function.*
- To use a function, you have **to *call* or *invoke*** it.
- There are two ways to call a function, depending on whether or not it returns a value.
- If the function returns a value, a call to that function is usually treated as a value. For example,

```
int larger = max(3, 4);
```

- calls max(3, 4) and assigns the result of the function to the variable larger.

CALLING A FUNCTION

- Another example of such a call is:

  cout << max(3, 4);

  which prints the return value of
  the function call max(3, 4).

- When the max function is invoked, the flow of control transfers to its definition. Once the max function is finished, it returns control to the caller.

CALLING A FUNCTION

```
#include <iostream>
using namespace std;

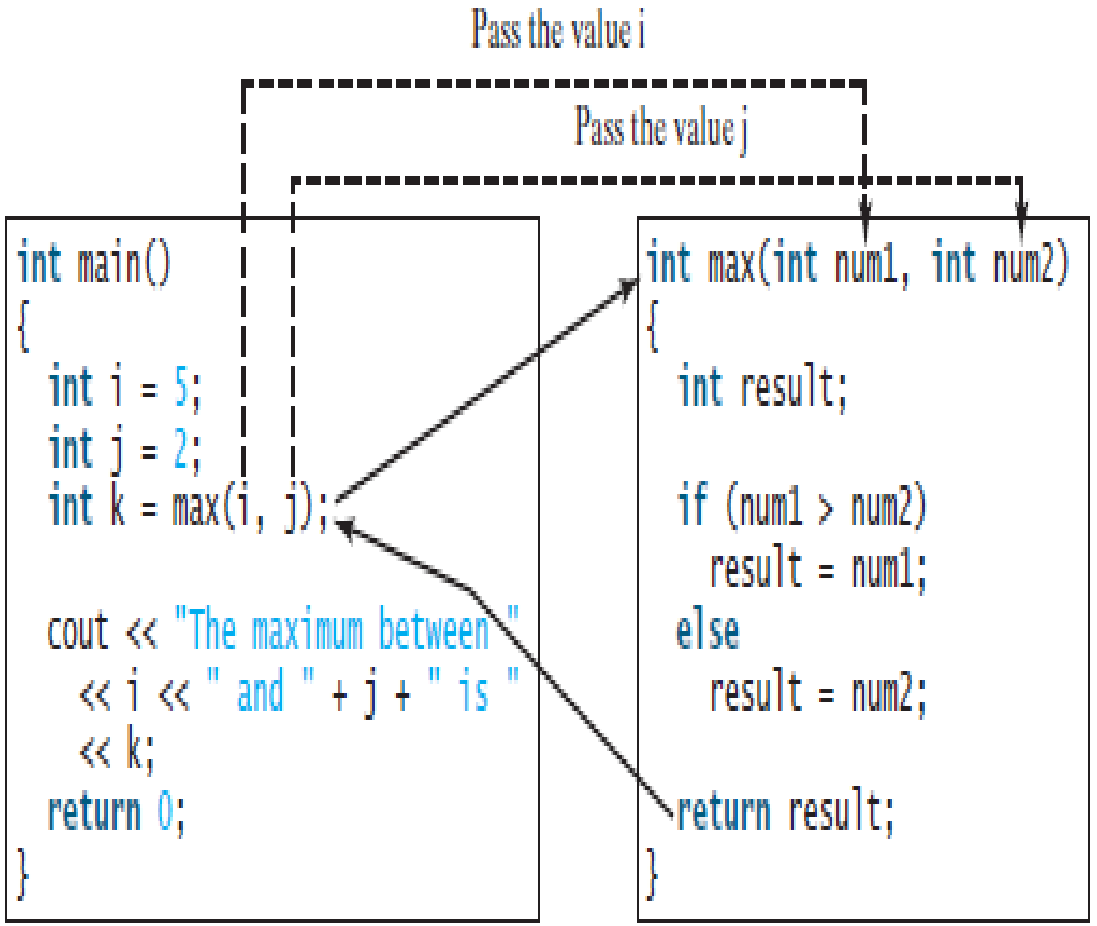
// Return the max between two numbers
int max(int num1, int num2)
{
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}

int main()
{
  int i = 5;
  int j = 2;
  int k = max(i, j);
  cout << "The maximum between " << i <<
    " and " << j << " is " << k << endl;

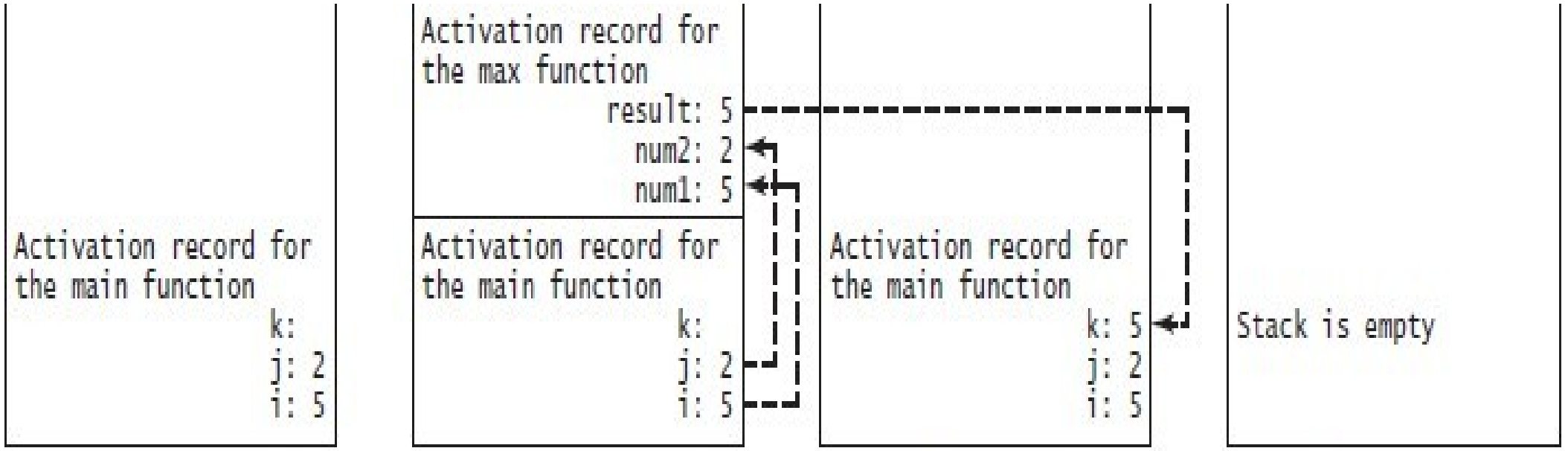
  return 0;
}
```



ACTIVATION RECORD

- Each time a function is invoked, the system creates an *activation record* (also called an *activation frame*) that stores its arguments and variables for the function and places the activation record in an area of memory known as a *call stack*.
- Stack is Last in First out (LIFO).
- When a function calls another function, the caller's activation record is kept intact and a new activation record is created for the new function call.
- When a function finishes its work and returns control to its caller, its activation record is removed from the call stack.

ACTIVATION RECORD



(a) The main function is invoked.

(b) The max function is invoked.

(c) The max function is finished and the return value is sent to k.

(d) The main function is finished.

a) Function with no return type and no arguments

MORE ABOUT FUNCTIONS

```
void sum()
{
 int a = 5;
 int b = 6;
 int sum = a+b;
 cout<<" sum = "<<sum<<endl;
}
```

```
main()
{
cout<<" function calling"<<endl;
sum();
cout<<" Program is going to end";
}
```

MORE ABOUT FUNCTIONS

b) Function with a return type but no arguments

```
int sum()
        {
         int a = 5;
         int b = 6;
         int sum = a+b;
        return sum;
         }
```

```
main()
{
cout<<" function calling"<<endl;
int add = sum();
cout<<" sum = "<<add<<endl;
}
```

MORE ABOUT FUNCTIONS

c) Function with no return type while having arguments.

```
void  sum (int a, int b)
{
    int sum = a+b;
    cout<<" sum = "<<sum;
}
```

```
main()
{
int c = 5;
int d = 6;
sum(c , d);
cout<<" Comsats "<<endl;
}
```

MORE ABOUT FUNCTIONS

d) Function with both return type and arguments.

```
int sum(int a, int b)
{
 int res = a + b;
 return res;
}
```

```
 main()
{
 int n1 = 8;
 int n2 = 3;
 int sum = sum(n1,n2);
 cout<<" sum = "<<sum;
}
```

FUNCTION PROTOTYPES

- *A* function prototype *declares a function without having to implement it.*
- Before a function is called, its header must be declared.
- One way to ensure this is to place the definition before all function calls.
- Another approach is to define a function prototype before the function is called. A function prototype, also known as *function declaration*, is a function header without implementation. The implementation is given later in the program.

FUNCTION PROTOTYPE

```
#include<iostream>
using namespace std;

void sum()
{
    int a = 5;
    int b = 6;
    int sum = a + b;
    cout<<" sum = "<<sum<<endl;
}

main()
{
sum();

}
```

No need to declare a function

```
#include<iostream>
using namespace std;

void sum();

main()
{
sum();

}
void sum()
{
    int a = 5;
    int b = 6;
    int sum = a + b;
    cout<<" sum = "<<sum<<endl;
}
```

Need to declare a function otherwise error will be generated that " sum is not decaled in this scope"

CLASS ACTIVITY 1

- Write function headers (not the bodies) for the following functions:
  a) Write a function prototype which takes two floating point numbers as arguments and return their product.
  b) Write a function prototype which takes a lower case alphabet as an argument and returns its equivalent upper case alphabet.
  c) Write a function prototype which takes a string and an integer as an argument and Displays that string a specified number of times provided in integer value.
  d) Display a message a specified number of times.

SOLUTION

a) float prod(float a , float b);

b) char alpha(char);

same as

char alpha(char
c);

c) void
display(string s,
int a);

CLASS ACTIVITY 2

Identify and correct the errors in the following program:

```
int function1(int n)
{
  cout << n;
}

function2(int n, m)
{
  n += m;
  function1(3.4);
}
```

HOME ACTIVITY

- What would be wrong with not writing a return statement in a value- returning function? Can you have a return statement in a void function? Does the return statement in the following function cause syntax errors?

```
void p(double x, double y)

{

cout << x << " " << y << endl; return x + y;

}
```

CONCLUSION

- Basics of functions
- Passing and returning values to functions

NEXT LECTURE

- Default Arguments
- Inline Functions
- Function Overloading
- Local, Global, and Static Local Variables